

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №3-4

«Функциональные возможности языка Python»

Выполнил:
студент группы ИУ5 - 32Б:
Акопов А.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Нардид А. Н.
Подпись и дата:

Москва, 2025 г.

Задание:

Задание лабораторной работы состоит из решения нескольких задач.

Файлы, содержащие решения отдельных задач, должны располагаться в пакете `lab_python_fr`. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

Задача 1 (файл `field.py`)

Необходимо реализовать генератор `field`. Генератор `field` последовательно выдает значения ключей словаря.

В качестве первого аргумента генератор принимает список словарей, дальше через `*args` генератор принимает неограниченное количество аргументов.

- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно `None`, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно `None`, то оно пропускается. Если все поля содержат значения `None`, то пропускается элемент целиком.

Код:

```
def field(lst, *args):
    if len(args) == 1:
        key = args[0]
        for item in lst:
            if key in item and item[key] is not None:
                yield item[key]
    else:
        for item in lst:
            result = {}
            for key in args:
                if key in item and item[key] is not None:
                    result[key] = item[key]
            if result:
                yield result

def test():
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
    ]
    print("Один аргумент:")
```

```

for value in field(goods, 'title'):
    print(value)

print("\nНесколько аргументов:")
for value in field(goods, 'title', 'price'):
    print(value)

```

```

● (base) an.akorov@MacBook-Air-Akorov lab3 % python3 main.py
Выберите номер задания от 1 до 7: 1
Один аргумент:
Ковер
Диван для отдыха

Несколько аргументов:
{'title': 'Ковер', 'price': 2000}
{'title': 'Диван для отдыха', 'price': 5300}

```

Задача 2 (файл gen_random.py)

Необходимо реализовать генератор gen_random(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона. Пример:

gen_random(5, 1, 3) должен выдать 5 случайных чисел в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

Код:

```

import random
def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

def print_gen():
    list_random = []
    for i in gen_random(5,1,3):
        list_random.append(i)
    print(*list_random)

```

```

● (base) an.akorov@MacBook-Air-Akorov lab3 % python3 main.py
Выберите номер задания от 1 до 7: 2
3 1 2 1 2
● (base) an.akorov@MacBook-Air-Akorov lab3 % python3 main.py
Выберите номер задания от 1 до 7: 2
2 1 3 2 3
● (base) an.akorov@MacBook-Air-Akorov lab3 % python3 main.py
Выберите номер задания от 1 до 7: 2
2 3 1 2 2
○ (base) an.akorov@MacBook-Air-Akorov lab3 %

```

Задача 3 (файл unique.py)

- Необходимо реализовать итератор Unique(данные), который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный bool-параметр ignore_case, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен False.
- При реализации необходимо использовать конструкцию **kwargs.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

Код:

```
from lab_python_fp.gen_random import gen_random

class Unique(object):
    def __init__(self, items, **kwargs):

        self.ignore_case = kwargs.get('ignore_case', False)
        self.items = iter(items)
        self.seen = set()
        self._first_iteration = True

    def __next__(self):
        while True:
            item = next(self.items)

            if self.ignore_case and isinstance(item, str):
                key = item.lower()
            else:
                key = item
            if key not in self.seen:
                self.seen.add(key)
                return item

    def __iter__(self):
        return self

def test_unique():
    print("Числа:")
    data1 = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
    for i in Unique(data1):
```

```

print(i, end=' ')

print("\n\nignore_case=False:")
data2 = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
for i in Unique(data2):
    print(i, end=' ')

print("\n\nignore_case=True ")
for i in Unique(data2, ignore_case=True):
    print(i, end=' ')

print("\n\nГенератор:")
data3 = gen_random(10, 1, 3)
for i in Unique(data3):
    print(i)

```

```

● (base) an.akopov@MacBook-Air-Akopov lab3 % python3 main.py
Выберите номер задания от 1 до 7: 3
Числа:
1 2

ignore_case=False:
a A b B

ignore_case=True
a b

Генератор:
3
1
2
○ (base) an.akopov@MacBook-Air-Akopov lab3 %

```

Задача 4 (файл sort.py)

Дан массив 1, содержащий положительные и отрицательные числа.

Необходимо **одной строкой кода** вывести на экран массив 2, которые содержит значения массива 1, отсортированные по модулю в порядке убывания.

Сортировку необходимо осуществлять с помощью функции sorted. Пример:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
```

```
Вывод: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
```

Необходимо решить задачу двумя способами:

1. С использованием lambda-функции.
2. Без использования lambda-функции.

Код:

```
def dates_print():
    data = [4, -30, 100, -100, 123, 1, 0, -1, -4]

    result = sorted(data, key = abs, reverse=True)
    print(result)

    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print(result_with_lambda)
```

```
● (base) an.akopov@MacBook-Air-Akopov lab3 % python3 main.py
Выберите номер задания от 1 до 7: 4
[123, 100, -100, -30, 4, -4, 1, -1, 0]
[123, 100, -100, -30, 4, -4, 1, -1, 0]
○ (base) an.akopov@MacBook-Air-Akopov lab3 %
```

Задача 5 (файл print_result.py)

Необходимо реализовать декоратор print_result, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (list), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (dict), то ключи и значения должны выводиться в столбик через знак равенства.

Код:

```
def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)

        if isinstance(result, list):
            for i in result:
                print(i)
        elif isinstance(result, dict):
            for key, value in result.items():
                print(f"{key} = {value}")
        else:
            print(result)
```

```

        return result
    return wrapper

@print_result
def test_1():
    return 1

@print_result
def test_2():
    return 'iu5'

@print_result
def test_3():
    return {'a': 1, 'b': 2}

@print_result
def test_4():
    return [1, 2]

def print_test():
    print('test_1')
    test_1()
    print('test_2')
    test_2()
    print('test_3')
    test_3()
    print('test_4')
    test_4()

```

```

● (base) an.akopov@MacBook-Air-Akopov lab3 % python3 main.py
Выберите номер задания от 1 до 7: 5
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2
○ (base) an.akopov@MacBook-Air-Akopov lab3 %

```

Задача 6 (файл cm_timer.py)

Необходимо написать контекстные менеджеры cm_timer_1 и cm_timer_2, которые считают время работы блока кода и выводят его на экран. Пример:

```
with cm_timer_1():  
    sleep(5.5)
```

После завершения блока кода в консоль должно вывестись time: 5.5 (реальное время может несколько отличаться).

cm_timer_1 и cm_timer_2 реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки contextlib).

Код:

```
import time  
from contextlib import contextmanager  
  
class cm_timer_1:  
    def __enter__(self):  
        self.start_time = time.time()  
        return self  
  
    def __exit__(self, exc_type, exc_val, exc_tb):  
        self.end_time = time.time()  
        self.elapsed_time = self.end_time - self.start_time  
        print(f"time: {self.elapsed_time:.1f}")  
  
@contextmanager  
def cm_timer_2():  
    start_time = time.time()  
    try:  
        yield  
    finally:  
        end_time = time.time()  
        elapsed_time = end_time - start_time  
        print(f"time: {elapsed_time:.1f}")  
  
def test_timer():  
    print("Testing cm_timer_1 (class-based):")  
    with cm_timer_1():  
        time.sleep(2.5)  
  
    print("\nTesting cm_timer_2 (contextlib-based):")
```

```
with cm_timer_2():  
    time.sleep(1.5)
```

```
● (base) an.akopov@MacBook-Air-Akopov lab3 % python3 main.py  
Выберите номер задания от 1 до 7: 6  
Testing cm_timer_1 (class-based):  
time: 2.5  
  
Testing cm_timer_2 (contextlib-based):  
time: 1.5  
○ (base) an.akopov@MacBook-Air-Akopov lab3 %
```

Задача 7 (файл process_data.py)

- В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.
- В файле [data_light.json](#) содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - f1, f2, f3, f4. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора @print_result печатается результат, а контекстный менеджер cm_timer_1 выводит время работы цепочки функций.
- Предполагается, что функции f1, f2, f3 будут реализованы в одну строку. В реализации функции f4 может быть до 3 строк.
- Функция f1 должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.
- Функция f2 должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию filter.
- Функция f3 должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python). Пример: Программист C# с опытом Python. Для модификации используйте функцию map.
- Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист C# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность — зарплата.

Код:

```
import json
```

```

import sys
import random
from lab_python_fp.unique import Unique
from lab_python_fp.print_result import print_result
from lab_python_fp.field import field
from lab_python_fp.cm_timer import cm_timer_1

path = sys.argv[1] if len(sys.argv) > 1 else 'data_light.json'

with open(path, 'r', encoding='utf-8') as f:
    data = json.load(f)

@print_result
def f1(arg):
    # Сортируем список профессий без повторений, игнорируя регистр
    return sorted(Unique(field(arg, 'job-name'), ignore_case=True), key=str.lower)

@print_result
def f2(arg):
    # Фильтруем только профессии программистов
    return list(filter(lambda x: x.lower().startswith('программист'), arg))

@print_result
def f3(arg):
    # Добавляем "с опытом Python" к каждой профессии
    return list(map(lambda x: f"{x} с опытом Python", arg))

@print_result
def f4(arg):
    # Генерируем зарплаты и объединяем с профессиями
    salaries = [f"зарплата {random.randint(100000, 200000)} руб." for _ in arg]
    return list(zip(arg, salaries))

def process():
    with cm_timer_1():
        f4(f3(f2(f1(data))))

```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ ТЕРМИНАЛ ... + v zsh - lab3 [] []

Продавец торгового зала
продавец-кассир
Продавец-консультант
Продавец-консультант бытовой техники
продавец-консультант-кассир
ПРОЕКТ-МЕНЕДЖЕР
Производитель работ
Производитель работ (прораб)
Промоутер
прораб
Прораб технологических трубопроводов
просмотрщик продукции медицинского назначения
Профконсультант
Прядильщик
прядильщица
психиатр-нарколог
Психиатр-нарколог детский
психолог
Психолог 0,5 ставки
Работа для студента
Работник в гостиницу
Работник на корпусную мебель
Работник склада
Рабочие в цех
Рабочий
Рабочий в автомастерскую
Рабочий зеленого хозяйства
Рабочий кормосклада
рабочий на мембранно-вакуумный пресс
Рабочий на плотницкие работы
Рабочий по комплексному обслуживанию и ремонту зданий
Рабочий по комплексному обслуживанию и ремонту зданий (Можайский филиал)
Рабочий по комплексному обслуживанию и ремонту зданий на 0,85 ставки
Рабочий по комплексному обслуживанию и ремонту здания
Рабочий по комплексному ремонту зданий 4 разряда
Рабочий по стирке и ремонту специальной одежды
Рабочий по уходу за животными
Рабочий пожарно-химической станции
Рабочий строительный
Рабочий сцены
Рабочий цеха
Рабочий цеха переработки
Рабочий-строитель
Радиомеханик по ремонту радиоэлектронного оборудования
Радиомонтажник
Разметчик

элбец 1 Пробелов: 4 UTF-8 LF { } Python [] Завершить настройку 3.13.5 (venv)

```
ПРОБЛЕМЫ  ВЫХОДНЫЕ ДАННЫЕ  ТЕРМИНАЛ  ...  + v  zsh - lab3  [ ] [ ] ... | [ ] X

Электросварщик ручной сварки
Электросварщики ручной сварки
Электрослесарь (слесарь) дежурный и по ремонту оборудования, старший
Электрослесарь по ремонту и обслуживанию автоматики и средств измерений электростанций
Электрослесарь по ремонту оборудования в карьере
Электроэрозионист
Эндокринолог
Энергетик
Энергетик литейного производства
Энтомолог
Юрисконсульт
юрисконсульт 2 категории
Юрисконсульт. Контрактный управляющий
Юрист
Юрист (специалист по сопровождению международных договоров, английский – разговорный)
Юрист волонтер
Юристконсульт
Программист
Программист / Senior Developer
Программист 1C
Программист C#
Программист C++
Программист C++/C#/Java
Программист/ Junior Developer
Программист/ технический специалист
Программист-разработчик информационных систем
Программист с опытом Python
Программист / Senior Developer с опытом Python
Программист 1C с опытом Python
Программист C# с опытом Python
Программист C++ с опытом Python
Программист C++/C#/Java с опытом Python
Программист/ Junior Developer с опытом Python
Программист/ технический специалист с опытом Python
Программист-разработчик информационных систем с опытом Python
('Программист с опытом Python', 'зарплата 105456 руб.')
('Программист / Senior Developer с опытом Python', 'зарплата 171721 руб.')
('Программист 1C с опытом Python', 'зарплата 192921 руб.')
('Программист C# с опытом Python', 'зарплата 154964 руб.')
('Программист C++ с опытом Python', 'зарплата 103740 руб.')
('Программист C++/C#/Java с опытом Python', 'зарплата 196697 руб.')
('Программист/ Junior Developer с опытом Python', 'зарплата 191718 руб.')
('Программист/ технический специалист с опытом Python', 'зарплата 130979 руб.')
('Программист-разработчик информационных систем с опытом Python', 'зарплата 172657 руб.')
time: 0.0
(base) an.akopov@MacBook-Air-Akopov lab3 %
```

Файл main.py

```
from lab_python_fp.field import test
from lab_python_fp.gen_random import print_gen
from lab_python_fp.unique import test_unique
from lab_python_fp.sort import dates_print
from lab_python_fp.print_result import print_test
from lab_python_fp.cm_timer import test_timer
from lab_python_fp.process_data import process

def main():
    tasks = [test, print_gen, test_unique, dates_print, print_test, test_timer, process]

    try:
        number = int(input('Выберите номер задания от 1 до 7: '))
        return tasks[number - 1]()
    except (IndexError, ValueError):
        print("Неверный номер задания")
        return
```

```
if __name__ == "__main__":  
    main()
```